

EXP 4: Exploration vs. Exploitation Analysis using Multi-Armed Bandit Algorithms

Aim

To analyze the exploration–exploitation trade-off using the Multi-Armed Bandit problem and compare how different strategies select the best-performing action.

Algorithm

- Initialize multiple arms with unknown reward probabilities.
- Set an exploration probability .
- For each round:
 - Generate a random number.
 - If the number is less than , explore by selecting a random arm.
 - Otherwise, exploit by selecting the arm with the highest estimated reward.
 - Observe the reward from the selected arm.
 - Update the estimated reward of that arm.
- Repeat the process for a fixed number of rounds.
- Calculate the total reward and compare exploration and exploitation performance.
- Identify the arm with the highest estimated reward.

CODE

```
import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

arms = 3

true_rewards = [1.0, 3.0, 2.0] counts =

np.zeros(arms)

estimated_rewards = np.zeros(arms) epsilon =

0.1

reward_history = []

average_reward_history = []

estimated_history = []

total_reward = 0

for trial in range(1, 301):

    if np.random.random() < epsilon:

        action = np.random.randint(arms) else:

            action = np.argmax(estimated_rewards)
```

```

reward=np.random.normal(true_rewards[action], 1)
counts[action] += 1
estimated_rewards[action] += ( reward -
    estimated_rewards[action]
) / counts[action]
total_reward += reward
reward_history.append(reward)
average_reward_history.append(
    total_reward / trial
)
estimated_history.append(
    estimated_rewards.copy()
)
estimated_history=np.array(estimated_history) print("=" *
55)
print("    MULTI-ARMED BANDIT")
print("=" * 55)
print("\nActual Average Rewards:")
for i in range(arms):
    print(
        "Arm", i + 1, ":",
        true_rewards[i]
    )

print("\nEstimated Rewards:")

for i in range(arms):
    print(
        "Arm", i + 1, ":",
        round(estimated_rewards[i], 2)

```

)

```
print("\nArm Selection Count:")
```

```
for i in range(arms): print(
```

```
    "Arm", i + 1, ":",
```

```
    int(counts[i])
```

```
)
```

```
best_arm = np.argmax(estimated_rewards) + 1
```

```
print("\nBest Arm:", best_arm)
```

```
print(
```

```
    "Final Average Reward:",
```

```
    round(average_reward_history[-1], 2)
```

```
)
```

```
plt.figure(figsize=(9, 5)) plt.plot(
```

```
    estimated_history[:, 0],
```

```
    label="Arm 1"
```

```
)
```

```
plt.plot(
```

```
    estimated_history[:, 1],
```

```
    label="Arm 2"
```

```
)
```

```
plt.plot(
```

```
    estimated_history[:, 2],
```

```
    label="Arm 3"
```

```
)
```

```

plt.xlabel("Number of Trials")
plt.ylabel("Estimated Reward")

plt.title("Exploration vs Exploitation")

plt.legend()

plt.grid(True)

plt.tight_layout() plt.show()

x=np.arange(arms) width
= 0.35

plt.figure(figsize=(8, 5)) plt.bar(
    x - width / 2,
    true_rewards,
    width,
    label="Actual Reward"
)

plt.bar(
    x + width / 2,
    estimated_rewards,
    width,
    label="Estimated Reward"
)

plt.xlabel("Arms")
plt.ylabel("Reward")

plt.title("Actual vs Estimated Reward Comparison")

plt.xticks(
    x,
    ["Arm 1", "Arm 2", "Arm 3"]
)

plt.legend()

plt.grid(axis="y", linestyle="--", alpha=0.5)

```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT

=== MULTI-ARMED BANDIT

Actual Average Rewards: Arm

1 : 1.0

Arm 2 : 3.0

Arm 3 : 2.0

Estimated Rewards: Arm

1 : 0.96

Arm 2 : 2.97

Arm 3 : 2.08

Arm Selection Count:

Arm 1 : 15

Arm 2 : 274

Arm 3 : 11

Best Arm: 2

Final Average Reward: 2.84

